

Tập 3: Kube-proxy & Bài toán Services

Lý thuyết: EndpointSlice, iptables chains, IPVS & nftables mode

Network Thực Chiến · Series: Kubernetes Networking · Tập 03

Services là gì? Bài toán cần giải

Pod có IP động — Pod chết đi, IP đó mất. Làm sao để client tìm được Pod?

```
Client → ??? → Pod A (10.244.1.5) [đang chạy]
                  Pod B (10.244.1.6) [đang chạy]
                  Pod C (10.244.2.3) [vừa chết, IP mất]
```

Service cung cấp một IP ảo ổn định (**ClusterIP**) và tên DNS cố định, đứng trước các Pod:

```
Client → ClusterIP: 10.96.50.100 → kube-proxy → Pod A / Pod B
```

Kube-proxy: Người dịch ClusterIP thành Pod IP

`kube-proxy` chạy trên **mọi Node**, theo dõi thay đổi từ API Server và lập trình các rule vào Linux kernel để thực hiện load balancing:

```
API Server
| Thông báo Service/EndpointSlice mới
▼
kube-proxy (trên mỗi Node)
| Lập trình rules vào:
| — iptables (mode mặc định cũ)
| — IPVS (mode hiệu năng cao)
| — nftables (mode mới, GA từ K8s v1.33)
```

EndpointSlice thay thế Endpoints (K8s v1.33)

`Endpoints` là tài nguyên cũ lưu danh sách IP của tất cả Pod sau Service. Vấn đề:

- Nếu Service có **1000 pods**, toàn bộ 1000 IP phải nằm trong **1 object** → Object khổng lồ, cập nhật chậm, tốn băng thông etcd.

EndpointSlice giải quyết bằng cách chia nhỏ:

```
Service: my-app
├── EndpointSlice-abc (max 100 pods/slice)
├── EndpointSlice-def (max 100 pods/slice)
└── EndpointSlice-xyz (max 100 pods/slice)
```

Từ K8s v1.33, `Endpoints` API cũ bị **deprecated** hoàn toàn.

iptables mode: Cơ chế DNAT

Khi bạn tạo Service `ClusterIP: 10.96.50.100`, kube-proxy tạo iptables rule:

```
# Chain KUBE-SERVICES là điểm vào
-A KUBE-SERVICES -d 10.96.50.100/32 -j KUBE-SVC-XXXXX

# Chain KUBE-SVC-XXXXX thực hiện load balancing bằng random module
-A KUBE-SVC-XXXXX -m statistic --mode random --probability 0.33 -j KUBE-SEP-AAAA
-A KUBE-SVC-XXXXX -m statistic --mode random --probability 0.5 -j KUBE-SEP-BBBB
-A KUBE-SVC-XXXXX -j KUBE-SEP-CCCC

# Chain KUBE-SEP-AAAA thực hiện DNAT đến Pod IP thực
-A KUBE-SEP-AAAA -j DNAT --to-destination 10.244.1.5:8080
```

IPVS mode: Tại sao hiệu năng cao hơn?

iptables duyệt rules theo **danh sách tuần tự** → $O(n)$ → Với 10,000 Services: cực chậm.

IPVS (IP Virtual Server) dùng **hash table** trong kernel → $O(1)$:

```
# Xem bảng IPVS
ipvsadm -Ln

# Output:
# TCP 10.96.50.100:80 rr
#   -> 10.244.1.5:8080   Round-Robin
#   -> 10.244.1.6:8080   Round-Robin
#   -> 10.244.2.3:8080   Round-Robin
```

Thêm vào đó, IPVS hỗ trợ nhiều thuật toán LB hơn: **rr**, **lc**, **dh**, **sh**, **sed**, **nq**.

externalTrafficPolicy: Ấn mình quan trọng

Policy	Luồng traffic NodePort	Đặc điểm
Cluster (default)	Đến bất kỳ Node nào → kube-proxy forward đến Pod bất kỳ	Phân phối đều, nhưng mất source IP (SNAT)
Local	Chỉ forward đến Pod trên chính Node đó	Giữ source IP , nhưng có thể mất cân bằng tải

Use case thực tế: Khi bạn cần biết IP thực của client (WAF, Rate-limiting theo IP), hãy dùng `externalTrafficPolicy: Local`.

nftables mode: Tương lai của kube-proxy

`nftables` là người kế nhiệm của `iptables` trong Linux kernel, GA trong K8s v1.33:

```
# Bật nftables mode trong kube-proxy ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: kube-proxy
data:
  config.conf: |
    mode: nftables  # ← Thêm dòng này
```

Ưu điểm: Cú pháp rõ ràng hơn, atomic rule update (không bị race condition), hiệu năng tốt hơn với tập rule lớn.

Tổng kết Tập 3

Khái niệm	Tóm tắt
ClusterIP	IP ảo ổn định đại diện cho nhóm Pod phía sau
kube-proxy	Lập trình iptables/IPVS/nftables để forward traffic đến Pod
EndpointSlice	Thay thế Endpoints, chia nhỏ danh sách Pod để scale tốt hơn
IPVS vs iptables	IPVS $O(1)$ vs iptables $O(n)$, phù hợp cluster lớn
externalTrafficPolicy	Local giữ source IP, Cluster phân tải đồng đều

👉 Chuyển sang Lab 1.3

Mở file `lab-guide.md` trong thư mục `1.3/` để thực hành:

- Tạo Deployment + Service và phân tích iptables chains
- Chuyển sang IPVS mode và dùng `ipvsadm`
- Thử nghiệm `externalTrafficPolicy`